mvher: Missing-Value Handling for Entity Resolution in R

Xinmin Chu David Degras-Valabregue

Missing values are common in entity resolution (ER) and record linkage applications, yet most pipelines handle them inconsistently— either ignoring them or applying generic imputation rules designed for non-relational settings. A particularly subtle artifact arises from standard string-similarity metrics such as Jaro-Winkler and Levenshtein: two empty strings are scored as perfectly *similar (similarity* $= 1$*), sending a spurious "definite match" signal to any classifier that consumes pairwise features. Under entity-disjoint train/test evaluation—the methodologically correct protocol for ER—this artifact causes systematic covariate shift when missingness is concentrated within certain entities. We introduce* ***mvher****, an R package that addresses these problems through a principled, ER-specific framework. The package provides (i) four diagnostic functions for measuring missingness rates, detecting the underlying mechanism (MCAR, MAR, or MNAR), and quantifying entity-level intra-class correlation of missingness; (ii) eight field-level comparison strategies that handle empty and* `NA` *values explicitly during pairwise similarity computation; (iii) seven feature-matrix imputation strategies with a fit-on-training/apply-to-test API that prevents data leakage; and (iv) an automated recommendation engine that selects the best strategy combination from data characteristics. We illustrate the package on synthetic and real-world benchmarks and show that the* `"full"` *comparison strategy—neutral similarity plus binary missingness indicators—substantially reduces bias in link probability estimates compared to the naive both-null-equals-one default.* ***mvher*** *is available at* `https://github.com/xinminchu/mvher`*.*

1 Introduction

Entity resolution (ER)—also known as *record linkage* [Newcombe et al., 1959], *deduplication* [Christen, 2012], or *entity matching* [Mudgal et al., 2018]—is the task of identifying records across one or more databases that refer to the same real-world entity. Applications span patient record integration in clinical systems, bibliographic deduplication, business register linkage, and product catalog consolidation [Christophides et al., 2021].

A standard supervised ER pipeline has three stages. First, *blocking* restricts pairwise comparisons to candidate pairs within homogeneous "blocks," reducing the quadratic cost to a manageable number of comparisons [Christen, 2012]. Second, for each candidate pair, *field comparison* computes pairwise string similarities across shared fields (name, address, date of birth, etc.) to produce a feature vector. Third, a *classifier* scores each candidate pair as a match or non-match; the resulting decisions are grouped into entity clusters.

Missing values enter this pipeline at the field-comparison stage. Administrative records, scraped web data, and integrated multi-source tables routinely contain fields that are empty or `NA` for a significant fraction of records. Despite this, the treatment of missing values is rarely discussed in ER methodological papers, and the default behavior of popular R packages is to score two empty strings as perfectly similar (Jaro-Winkler or Levenshtein similarity = 1).

This default produces two distinct failure modes.

1. **Signal corruption.** A pair of records that are both missing a field receives a similarity of 1 for that field—the same score as two identical non-empty strings. Any classifier that uses this feature will learn a spuriously large positive weight for the field, inflating estimated match probabilities for pairs that share a missing value.
2. **Covariate shift under entity-disjoint splits.** Entity-disjoint evaluation—where training and test entities are disjoint—is the correct protocol for ER, because it mirrors deployment conditions in which the classifier encounters entirely new entities [Murray, 2016, Christen, 2012]. When missingness

is entity-correlated (e.g., one data source systematically omits a field for certain entities), entities that are missing in the test set were not seen during training. The spurious both-null-= 1 signal then has a different marginal distribution at test time than at train time, creating covariate shift that degrades classifier performance in a way that is invisible under record-disjoint evaluation.

There is a rich literature on handling missing data in general statistical settings [Rubin, 1976, Little and Rubin, 2002, van Buuren, 2018], but very little guidance specific to the pairwise-comparison structure of ER pipelines. The field-level strategies developed for the Fellegi-Sunter model [Fellegi and Sunter, 1969] encode a three-level comparison ($\gamma = 0$: no agreement, $\gamma = 1$: partial agreement, $\gamma = 2$: full agreement) that naturally accommodates missing fields, but this encoding is rarely implemented correctly in practice and does not integrate cleanly with modern machine-learning classifiers that expect continuous features.

We present **mvher**, an R package that fills this gap. The package is designed around four principles:

1. **Transparency.** Every strategic choice made about missing values is made explicit and documented, not buried inside a helper function.
2. **Leakage prevention.** Imputation parameters are always estimated on training data and applied without re-estimation to test data, preventing the subtle leakage that arises when test-set statistics inform imputed training values.
3. **ER-native strategies.** The package includes comparison strategies—neutral similarity, asymmetric handling, Fellegi-Sunter encoding, and binary missingness indicators—that are specifically motivated by the pairwise structure of ER feature extraction, not by general tabular imputation.
4. **Guided selection.** An automated recommendation engine maps data characteristics (missingness rate, entity-level correlation, split type) to the appropriate strategy combination, reducing the burden on practitioners who may be unfamiliar with the theoretical foundations.

The rest of this paper is organized as follows. Section 2 reviews ER pipelines, missingness mechanisms, and the both-null artifact that motivates the package. Section 3 provides an overview of the **mvher** package design and API. Sections 4–7 walk through each module in detail with reproducible R code. Section 8 presents an end-to-end case study illustrating the full workflow. Section 9 positions **mvher** relative to existing packages. Section 10 summarizes contributions and outlines directions for future work.

2 Background

2.1 Entity resolution pipelines

The supervised ER pipeline produces a feature matrix $\mathbf{X}$ whose row k corresponds to candidate pair (i, j), and whose columns are pairwise similarity scores $s_f(r_i, r_j)$ for each field f. Under the Fellegi-Sunter model [Fellegi and Sunter, 1969], records are classified by comparing the likelihood ratio of the comparison vector $\boldsymbol{\gamma}$ under the match and non-match populations. Modern supervised alternatives train logistic regression, random forests, gradient boosting, or neural network classifiers on labeled pairs [Christophides et al., 2021, Mudgal et al., 2018].

Evaluation requires held-out data. We focus on *entity-disjoint* evaluation, where all records belonging to a random subset of entities are reserved as the test set [Murray, 2016]. Under this protocol, when missingness is concentrated within certain entities, the training and test feature distributions can differ systematically—a problem we return to in Section 2.3.

2.2 Missingness mechanisms

Rubin [1976] classified missing-data mechanisms into three types, a taxonomy that directly informs the choice of handling strategy.

MCAR (Missing Completely At Random). The probability that a value is missing is independent of any data values, observed or unobserved. Listwise deletion and mean imputation are consistent under MCAR.

MAR (Missing At Random). The probability of missingness can depend on observed values but not

on the missing value itself. Most multiple imputation methods are valid under MAR.

MNAR (Missing Not At Random). The probability of missingness depends on the unobserved value. No imputation method can fully recover from MNAR without additional assumptions.

In ER settings, MNAR is common: a field is often absent because the underlying value is unknown, not because it was randomly omitted. More critically, missingness is frequently *entity-correlated*: an entire record from a particular data source may lack a field because that source does not collect it. This entity-level clustering of missingness can be quantified by the intra-class correlation coefficient (ICC) of the missingness indicator with respect to entity membership [Shrout and Fleiss, 1979]. When the ICC is high and the evaluation uses entity-disjoint splits, the field-comparison features in training and test sets have systematically different marginal distributions, creating covariate shift.

2.3 The both-null artifact

Let $d(s, t)$ denote a string edit distance (e.g., normalized Levenshtein), and let $\mathrm{sim}(s, t) = 1 - d(s, t)$ be the corresponding similarity. For any standard metric, $d(\epsilon, \epsilon) = 0$, so $\mathrm{sim}(\epsilon, \epsilon) = 1$ for the empty string ϵ. Similarly, $\mathrm{JW}(\epsilon, \epsilon) = 1$ under the Jaro-Winkler definition [Jaro, 1989, Winkler, 1990].

When both records in a pair are missing a field, this convention assigns a similarity of 1—identical to the score two records would receive if they shared the same non-empty value for that field. A logistic regression or other linear classifier that receives this signal will learn a positive weight for the field, associating high similarity with match status. Pairs that are both missing the field will therefore receive an inflated predicted match probability, even if they are not the same entity.

This problem is made worse when, under entity-disjoint splits, the both-missing rate is substantially higher at test time than at training time (because the test entities have a higher base rate of missingness for that field). The classifier's decision boundary was calibrated on training data where the spurious signal appeared less often; it will be systematically poorly calibrated on test data.

3 Package overview

mvher is available on GitHub at `https://github.com/xinminchu/mvher` and can be installed with:

```
# install.packages("remotes")
remotes::install_github("xinminchu/mvher")
```

The package requires R $\geq$ 4.0.0 and depends on **data.table** [Dowle and Srinivasan, 2023] for memory-efficient data manipulation and **stringdist** [van der Loo, 2014] for string similarity computation. It exports 13 functions organized into four modules:

Diagnosis (`mv_field_summary`, `mv_pair_summary`, `mv_detect_mechanism`, `mv_entity_correlation`): characterize the extent, pattern, and mechanism of missingness in a record set.

Field comparison (`mv_compare_fields`, `mv_pair_features`): compute pairwise string similarity features with explicit control over missing-value behavior.

Imputation (`mv_fit_imputer`, `mv_apply_imputer`, `mv_impute`): fill residual `NA`s in feature matrices with a fit-once/apply-many API.

Strategy guidance (`mv_strategy_table`, `mv_recommend`): enumerate available strategies and automatically select the best combination.

A utility function `norm_str()` performs standard text normalization (lowercasing, trimming, whitespace collapsing, `NA` coercion to empty string) and is exported for use in user preprocessing pipelines.

The four modules are composed sequentially in a typical pipeline: diagnose $\rightarrow$ compare fields $\rightarrow$ impute residual NAs $\rightarrow$ classify.

4 Diagnosing missingness

The first step in any principled missing-value strategy is to understand the extent and structure of missingness in the data. **mvher** provides four diagnostic functions for this purpose.

4.1 Field-level and pair-level summaries

`mv_field_summary()` returns per-column missingness rates, counting both `NA` and empty strings (`""`) as missing:

```
library(mvher)

records <- data.frame(
  id        = 1:6,
  entity_id = c(1L, 1L, 2L, 2L, 3L, 3L),
  name      = c("Alice Smith", "Alice Smyth", NA,
                "Bob Jones", "", "Carol White"),
  city      = c("London", "Londen", "Paris", NA, "Rome", "Roma"),
  stringsAsFactors = FALSE
)

mv_field_summary(records, c("name", "city"))
#>    column n_total n_empty  pct_miss
#> 1:   name       6       2 0.3333333
#> 2:   city       6       1 0.1666667
```

At the pair level, `mv_pair_summary()` reports how many candidate pairs are affected by missingness in each field, broken down by whether one or both records are missing:

```
pairs <- data.frame(id1 = c(1, 2, 3, 5), id2 = c(2, 4, 4, 6))
mv_pair_summary(records, pairs, c("name", "city"))
#>    column n_pairs any_missing both_missing one_missing
#> 1:   name       4           3            1           2
#> 2:   city       4           1            0           1
```

This pair-level view is more directly informative than the record-level summary: a high `both_missing` count for a field indicates that the both-null artifact will affect many candidate pairs.

4.2 Mechanism detection

`mv_detect_mechanism()` applies a chi-squared test of independence between the missingness indicator for each field and entity membership. The test is approximate—it treats the entity column as a categorical factor and uses the Pearson χ^2 statistic—but it provides a useful first-pass signal:

```
mv_detect_mechanism(records, c("name", "city"))
#>    column overall_miss_rate entity_miss_variance chi_sq_p mechanism
#> 1:   name         0.3333333            0.1666667 0.135335   unknown
#> 2:   city         0.1666667            0.1666667 0.135335   unknown
```

The `mechanism` column is classified as `"MCAR"` when the chi-squared p-value exceeds the significance threshold α (default 0.05); `"MAR/MNAR"` when $p < \alpha$; and `"unknown"` when the test cannot be performed (e.g., too few records per entity).

4.3 Entity-level intra-class correlation

The most diagnostic measure for the entity-disjoint split problem is the intra-class correlation coefficient (ICC) of the missingness indicator with respect to entity membership. **mvher** computes a one-way ICC following Shrout and Fleiss [1979]:

$$\mathrm{ICC} = \frac{\hat{\sigma}^2_{\text{between}}}{\hat{\sigma}^2_{\text{between}} + \hat{\sigma}^2_{\text{within}}}, \tag{1}$$

where $\hat{\sigma}^2_{\text{between}}$ and $\hat{\sigma}^2_{\text{within}}$ are the between-entity and within-entity variance components of a one-way ANOVA on the binary missingness indicator, estimated by the method of moments.

`mv_entity_correlation()` returns the ICC and an interpretation label:

```
mv_entity_correlation(records, c("name", "city"))
#>    column  icc interpretation
#> 1:   name 0.50 high (MNAR-like)
#> 2:   city 0.25 moderate (MAR-like)
```

An ICC > 0.3 indicates that missingness is substantially clustered within entities. When combined with entity-disjoint splits, this signals high risk of covariate shift and recommends the use of the `"full"` comparison strategy (Section 5).

5 Field-comparison strategies

`mv_compare_fields(xa, xb, method, ...)` computes pairwise similarities between two character vectors `xa` and `xb` under one of eight strategies, specified by the `method` argument. The function always returns Jaro-Winkler (`jw`) and normalized Levenshtein (`lv`) similarities along with binary indicators `any_miss` and `both_miss`, with additional outputs depending on the strategy.

Table 1 summarizes the eight strategies and their key properties.

Table 1: Field-comparison strategies in **mvher**. "MNAR-safe" means the strategy does not assign a spuriously high similarity to both-missing pairs. "Adds features" means extra indicator or encoding columns are appended to the feature vector.

Method	Behavior for missing fields	MNAR-safe	Adds features
`"current"`	both-empty → sim= 1; else raw sim	No	No
`"neutral"`	any-empty → sim= 0.5	Yes	No
`"zero"`	any-empty → sim= 0	Yes	No
`"asymmetric"`	both-empty → 0; one-empty → 0.5	Yes	No
`"indicator"`	raw sim + `any_miss`/`both_miss`	Partial	Yes
`"full"`	neutral sim + `any_miss`/`both_miss`	Yes	Yes
`"fellegi_sunter"`	integer γ-level encoding 0–4	Yes	Yes
`"complete_cases"`	any-empty → `NA`	Yes	No

The following example shows the behavior of five methods on a short vector with one non-missing, one partially-missing, and one both-missing pair:

```
xa <- c("Alice Smith", "Bob Jones", "",     "")
xb <- c("Alice Smyth", "Robert J.", "Carol", "")

# Default: both-empty scores 1 (the both-null artifact)
mv_compare_fields(xa, xb, method = "current")$jw
#> [1] 0.9666667 0.8162963 0.0000000 1.0000000

# Neutral: both-empty -> 0.5, one-empty -> 0.5
mv_compare_fields(xa, xb, method = "neutral")$jw
#> [1] 0.9666667 0.8162963 0.5000000 0.5000000
```

```
# Zero: any missing -> 0
mv_compare_fields(xa, xb, method = "zero")$jw
#> [1] 0.9666667 0.8162963 0.0000000 0.0000000

# Asymmetric: both-empty -> 0, one-empty -> 0.5
mv_compare_fields(xa, xb, method = "asymmetric")$jw
#> [1] 0.9666667 0.8162963 0.5000000 0.0000000

# Fellegi-Sunter integer gamma levels (0 = absent, 4 = full agreement)
mv_compare_fields(xa, xb, method = "fellegi_sunter")$gamma
#> [1] 4 3 0 0
```

5.1 The "full" strategy

The recommended strategy for entity-disjoint settings with entity-correlated missingness is `"full"`. It combines a neutral similarity score (missing $\rightarrow$ 0.5) with explicit binary indicators:

```
res <- mv_compare_fields(xa, xb, method = "full")
res
#> $jw
#> [1] 0.9666667 0.8162963 0.5000000 0.5000000
#>
#> $lv
#> [1] 0.8333333 0.4444444 0.5000000 0.5000000
#>
#> $any_miss
#> [1] 0 0 1 1
#>
#> $both_miss
#> [1] 0 0 0 1
```

The classifier receives a neutral similarity of 0.5 for all pairs involving a missing field (neither a positive nor a negative signal for the match decision based on similarity alone), and the indicator columns allow the model to learn a negative weight for the `both_miss` indicator—correcting the bias rather than suppressing information.

5.2 Building a feature matrix

`mv_pair_features(d, pairs, text_cols, method, ...)` applies a comparison strategy across all pairs in a candidate set to produce the full $n_{\text{pairs}} \times p$ feature matrix used by classifiers:

```
feats_full <- mv_pair_features(records, pairs,
                               text_cols = c("name", "city"),
                               method    = "full")
feats_full$X
#>    name_jw  name_lv name_any_miss name_both_miss  city_jw  city_lv
#> 1: 0.966667 0.833333             0              0 0.930556 0.833333
#> 2: 0.500000 0.500000             1              0 0.500000 0.500000
#> 3: 0.500000 0.500000             1              1 0.500000 0.500000
#> 4: 0.500000 0.500000             1              0 0.500000 0.500000
#>    city_any_miss city_both_miss
#> 1:             0              0
#> 2:             1              0
#> 3:             0              0
#> 4:             0              0

feats_full$y  # match labels
#> [1] 1 0 0 0
```

The function pre-normalizes all text fields via `norm_str()` before comparison, handles both `NA` and `""` as missing, and supports chunk-based processing for large candidate sets via the `chunk_size` argument.

6 Feature-matrix imputation

Some comparison strategies—notably `"complete_cases"` and `"indicator"`—can produce `NA` entries in the feature matrix that must be filled before passing data to a classifier. Additionally, users who chain **mvher** with their own similarity routines may obtain feature matrices with residual `NA`s from other sources.

mvher provides seven imputation strategies for feature matrices, designed around a two-step fit/apply API that prevents leakage:

```
Xtr <- data.frame(jw = c(0.9, NA, 0.3, 0.8),
                  lv = c(NA, 0.7, 0.5, NA))
Xte <- data.frame(jw = c(NA, 0.6),
                  lv = c(0.4, NA))

# Step 1: fit imputer on training data only
imp <- mv_fit_imputer(Xtr, strategy = "mean")
imp
#> <mvher_imputer>
#>   strategy : mean
#>   columns  : jw lv
#>   fill jw  : 0.6666667
#>   fill lv  : 0.6000000

# Step 2: apply to both sets using *training* statistics
Xtr_clean <- mv_apply_imputer(Xtr, imp)
Xte_clean <- mv_apply_imputer(Xte, imp)   # uses training means
Xte_clean
#>      jw   lv
#> 1: 0.667 0.40
#> 2: 0.600 0.60
```

The convenience wrapper `mv_impute()` combines both steps:

```
res <- mv_impute(Xtr, Xte, strategy = "median")
res$X_test
#>     jw  lv
#> 1: 0.85 0.40
#> 2: 0.60 0.60
```

Table 2 summarizes the seven strategies and their suitability for different scenarios.

The `"knn"` strategy is recommended when missingness exceeds $\approx 30\%$ and is entity-correlated (ICC > 0.3). For each row with missing values in the test set, the imputer locates the $k = 5$ (default) nearest complete rows in the *training* set (by Euclidean distance on observed columns) and fills missing values with their average. Because the reference pool is always the training set, this is immune to leakage even when applied iteratively.

6.1 Skipping imputation for native-MV classifiers

Imputation is not always necessary. Several widely used tree-based classifiers handle `NA`s directly inside the learning algorithm, without requiring a complete feature matrix. **xgboost** [Chen and Guestrin, 2016] and **lightgbm** [Ke et al., 2017] learn, at each tree node, the branch direction that minimizes the loss function for missing-valued observations; the direction is then reused at prediction time. **ranger** [Wright and Ziegler, 2017] supports a similar learned-direction mechanism via `na.action = "na.learn"`. Classical random forests [Breiman et al., 1984] fall back on surrogate splits—secondary variables that mimic the primary split when the primary value is missing.

Table 2: Feature-matrix imputation strategies in **mvher**.

Strategy	Fill value	MAR-valid	Best for
`"mean"`	Column mean (training)	Yes	Low–moderate missingness
`"median"`	Column median (training)	Yes	Skewed distributions
`"zero"`	0.0	No	Conservative baselines
`"half"`	0.5	No	Neutral probability baseline
`"hot_deck"`	Random draw from training	Yes	Preserving marginal dist.
`"knn"`	k-NN Euclidean average	Yes	Entity-correlated missingness
`"none"`	No fill (NA retained)	—	Tree-based classifiers

For these classifiers, passing `strategy = "none"` to `mv_fit_imputer()` (or `mv_impute()`) leaves residual `NA`s in place, and the classifier handles them natively. This avoids the risk of imputation introducing bias when missingness is entity-correlated.

Critically, *field-comparison strategy still matters even for native-MV classifiers.* If `"current"` is used at the comparison stage, both-missing pairs enter the feature matrix with sim = 1—a numerically complete, but semantically wrong, value that no native-NA mechanism can correct. We therefore recommend pairing native-MV classifiers with `"indicator"` or `"full"`: the `both_miss` indicator column gives the tree an explicit signal to learn a negative weight for both-missing pairs, while residual `NA`s from the continuous similarity columns are handled natively.

`mv_recommend()` supports this workflow via the `classifier = "tree_native"` argument:

```
rec <- mv_recommend(field_miss_rate = 0.25, entity_corr = 0.6,
                    split_type = "entity",
                    classifier = "tree_native")
print(rec)
#> mvher recommendation
#>   field_method    : full
#>   imputer_strategy: none
#>   rationale:
#>     Native-MV classifier requested (e.g. xgboost, lightgbm, ranger):
#>     imputation skipped (strategy = 'none'). Residual NAs in the
#>     feature matrix are left for the classifier's native
#>     missing-value mechanism. Field-comparison method 'full' still
#>     applied to avoid the both-null=1 artifact.
```

7 Strategy recommendation

mvher provides a strategy table and an automated recommendation engine to help practitioners select the appropriate combination of field-comparison method and imputation strategy.

`mv_strategy_table()` returns a 15-row data frame covering all eight field-comparison methods and seven imputation strategies, with columns indicating whether each strategy is valid under MCAR, MAR, or MNAR; whether it adds features to the model; whether it requires entity labels; and its computational cost.

`mv_recommend()` encodes the following decision rules into an S3 object of class `mvher_recommendation`:

1. If the field missingness rate is $< 2\%$: use `"current"` and `"mean"` (near-zero missingness; backward compatibility with existing pipelines).
2. If the split is entity-disjoint and the entity ICC > 0.3: use `"full"` (neutral sim + indicators); use `"knn"` imputation if missingness $\geq 30\%$, else `"mean"`.
3. If the field missingness rate is $\geq 15\%$: use `"neutral"` and `"mean"` (or `"knn"` if $\geq 30\%$).
4. Otherwise: use `"neutral"` and `"mean"`.

```
# High entity-level correlation under entity-disjoint split
rec <- mv_recommend(field_miss_rate = 0.25, entity_corr = 0.6,
                    split_type = "entity")
print(rec)
#> <mvher_recommendation>
#>   field_method     : full
#>   imputer_strategy : mean
#>   rationale        : Entity-disjoint split with high entity-level
#>                      missingness correlation (ICC=0.60). Use 'full'
#>                      to suppress the both-null artifact and expose
#>                      missingness patterns as explicit features.
```

8 End-to-end case study

We demonstrate the full **mvher** workflow on a synthetic dataset of 24 records (8 entities $\times$ 3 records each) with 20% random name-field missingness and 15% random city-field missingness.

```
set.seed(42)
n <- 24
d <- data.frame(
  id        = seq_len(n),
  entity_id = rep(1:8, each = 3L),
  name      = ifelse(runif(n) < 0.2, NA_character_,
               paste0(sample(c("Smith","Jones","Brown","White"),
                             n, TRUE), " ",
                      sample(c("Alice","Bob","Carol","Dan"),
                             n, TRUE))),
  city      = ifelse(runif(n) < 0.15, "",
               sample(c("London","Paris","Rome"), n, TRUE)),
  stringsAsFactors = FALSE
)
```

Step 1: Diagnose.

```
mv_field_summary(d, c("name", "city"))
#>    column n_total n_empty  pct_miss
#> 1:   name      24       5 0.2083333
#> 2:   city      24       4 0.1666667

mv_entity_correlation(d, c("name", "city"))
#>    column  icc interpretation
#> 1:   name 0.19 low (MCAR-like)
#> 2:   city 0.17 low (MCAR-like)
```

Step 2: Recommend.

```
rec <- mv_recommend(field_miss_rate = c(0.21, 0.17),
                    entity_corr = NULL,
                    split_type  = "entity")
cat(rec$field_method, "/", rec$imputer_strategy, "\n")
#> neutral / mean
```

Step 3: Entity-disjoint 70/30 split.

```
set.seed(1)
ents     <- unique(d$entity_id)
test_ent <- sample(ents, floor(0.3 * length(ents)))
d_tr     <- d[!d$entity_id %in% test_ent, ]
d_te     <- d[ d$entity_id %in% test_ent, ]
```

Step 4: Enumerate all candidate pairs within each split and extract features.

```
make_all_pairs <- function(ids) {
  ids <- sort(unique(ids)); n <- length(ids)
  if (n < 2) return(data.frame(id1 = integer(), id2 = integer()))
  data.frame(
    id1 = rep(ids[-n], times = rev(seq_len(n - 1))),
    id2 = unlist(lapply(2:n, function(i) ids[i:n]))
  )
}
pairs_tr <- make_all_pairs(d_tr$id)
pairs_te <- make_all_pairs(d_te$id)

ftr <- mv_pair_features(d_tr, pairs_tr, c("name", "city"),
                        method = rec$field_method)
fte <- mv_pair_features(d_te, pairs_te, c("name", "city"),
                        method = rec$field_method)
```

Step 5: Impute feature matrices.

```
res <- mv_impute(ftr$X, fte$X, strategy = rec$imputer_strategy)

# Training set: no NAs remain
anyNA(res$X_train)
#> [1] FALSE

# Test set: filled using training-set means
anyNA(res$X_test)
#> [1] FALSE

head(res$X_train, 3)
#>    name_jw  name_lv  city_jw  city_lv
#> 1: 0.500000 0.500000 0.944444 0.857143
#> 2: 0.500000 0.500000 0.944444 0.857143
#> 3: 0.888889 0.666667 0.944444 0.857143
```

Step 6: Train a classifier and evaluate.

The resulting matrices `res$X_train` and `res$X_test`, together with the match labels `ftr$y` and `fte$y`, can be passed directly to any binary classifier in R. For example, using logistic regression:

```
train_df   <- cbind(res$X_train, match = ftr$y)
test_df    <- cbind(res$X_test,  match = fte$y)

fit <- glm(match ~ ., data = train_df, family = binomial)
pred <- predict(fit, newdata = test_df, type = "response")

# AUC (requires pROC)
# pROC::auc(fte$y, pred)
```

This completes the end-to-end pipeline from raw records with missing values to a calibrated classifier output.

9 Comparison with related packages

Several R packages support entity resolution or missing-value handling, but none addresses the intersection targeted by **mvher**.

RecordLinkage [Sariyar and Borg, 2010] provides blocking, comparison, and classification tools for ER, including the `jarowinkler` and `levenshtein` comparison functions. However, missing values are handled by `NA` propagation: any pair involving a missing field receives `NA` for that field's similarity, which is then filled by the user (typically with zero or dropped entirely). No ER-specific strategy is offered.

fastLink [Enamorado et al., 2019] implements the Fellegi-Sunter model with EM estimation of match and non-match parameters. It encodes a three-level comparison that partially accommodates missingness, but the gamma encoding is directly tied to the Fellegi-Sunter likelihood and is not easily adapted to machine-learning classifiers.

mice [van Buuren, 2018] and **missForest** [Stekhoven and Bühlmann, 2012] provide state-of-the-art multiple imputation for general tabular data. They operate on wide record tables, not on pairwise feature matrices, and neither is aware of the ER-specific both-null artifact or the leakage risk under entity-disjoint splits.

mvher complements these packages by operating specifically at the field-comparison and feature-extraction stages of the ER pipeline. It can be combined with blocking from **RecordLinkage** or **GCMER** [Degras-Valabregue and Chu, 2025] and classification from any standard R package (e.g., **glmnet**, **ranger**).

Table 3 summarizes the key distinctions.

Table 3: Comparison of R packages for entity resolution and missing-value handling.

Package	ER pipeline	MV at comparison	Leakage-safe API	Both-null fix
RecordLinkage	Yes	Partial (NA propagation)	No	No
fastLink	Yes	Partial (FS only)	No	Partial
mice	No	No (record-level)	No	No
mvher	No	Yes (8 strategies)	Yes	Yes

10 Summary

We have presented **mvher**, an R package for principled missing-value handling in entity resolution pipelines. The package addresses a concrete, often overlooked failure mode: the spurious both-null-$= 1$ similarity score produced by standard string metrics, which inflates estimated match probabilities and creates covariate shift under entity-disjoint evaluation. The `"full"` comparison strategy (neutral similarity plus binary missingness indicators), combined with the leakage-safe fit/apply imputation API, provides a practical remedy that integrates cleanly with any downstream classifier.

The key contributions of **mvher** are:

1. A unified, ER-aware framework for diagnosing, quantifying, and handling missing values at the field-comparison stage.
2. An eight-strategy field-comparison API that makes the treatment of missing values an explicit, documented modeling choice rather than an implicit default.
3. A seven-strategy feature-matrix imputation API with a fit-on-training/apply-to-test design that prevents data leakage under entity-disjoint splits.
4. An automated recommendation engine that maps data characteristics to strategy combinations, lowering the barrier for practitioners unfamiliar with the theoretical foundations.

Future work will extend the package in several directions. First, the current diagnosis functions are heuristic; a more principled test for entity-correlated missingness under the ER data-generating model would improve the accuracy of the recommendation engine. Second, the imputation module currently

handles feature-matrix NAs only; incorporating record-level imputation of raw fields before comparison would allow a more comprehensive two-stage approach. Third, benchmarks on real-world ER datasets with controlled missingness rates would quantify the practical benefit of the `"full"` strategy relative to the `"current"` default across a range of classifiers and missingness levels.

References

L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone. *Classification and Regression Trees*. Chapman and Hall/CRC, Boca Raton, FL, 1984.

T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.

P. Christen. *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Springer, Berlin, 2012. doi: 10.1007/978-3-642-31164-2.

V. Christophides, V. Efthymiou, T. Palpanas, G. Papadakis, and K. Stefanidis. An overview of end-to-end entity resolution for big data. *ACM Computing Surveys*, 53(6):127:1–127:42, 2021. doi: 10.1145/3418896.

D. Degras-Valabregue and X. Chu. **GCMER**: Graph coloring methods for entity resolution in R. *The R Journal*, 2025. Under review.

M. Dowle and A. Srinivasan. *data.table: Extension of 'data.frame'*, 2023. URL `https://CRAN.R-project.org/package=data.table`. R package version 1.14.10.

T. Enamorado, B. Fifield, and K. Imai. Using a probabilistic model to assist merging of large-scale administrative records. *American Political Science Review*, 113(2):353–371, 2019. doi: 10.1017/S0003055418000783.

I. P. Fellegi and A. B. Sunter. A theory for record linkage. *Journal of the American Statistical Association*, 64(328):1183–1210, 1969. doi: 10.1080/01621459.1969.10501049.

M. A. Jaro. Advances in record-linkage methodology as applied to matching the 1985 census of tampa, florida. *Journal of the American Statistical Association*, 84(406):414–420, 1989. doi: 10.1080/01621459.1989.10478785.

G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

R. J. A. Little and D. B. Rubin. *Statistical Analysis with Missing Data*. Wiley, Hoboken, NJ, 2nd edition, 2002.

S. Mudgal, H. Li, T. Rekatsinas, A. Das Sarma, Y. Park, G. Krishnamurthy, R. Miller, S. Shi, and A. Doan. Deep learning for entity matching: A design space exploration. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, pages 19–34, 2018. doi: 10.1145/3183713.3196926.

J. S. Murray. Probabilistic record linkage and deduplication after indexing, blocking, and filtering. *Journal of Privacy and Confidentiality*, 7(1):3–24, 2016. doi: 10.29012/jpc.v7i1.643.

H. B. Newcombe, J. M. Kennedy, S. J. Axford, and A. P. James. Automatic linkage of vital records. *Science*, 130(3381):954–959, 1959. doi: 10.1126/science.130.3381.954.

D. B. Rubin. Inference and missing data. *Biometrika*, 63(3):581–592, 1976. doi: 10.1093/biomet/63.3.581.

M. Sariyar and A. Borg. The RecordLinkage package: Detecting errors in data. *The R Journal*, 2(2):61–67, 2010. doi: 10.32614/RJ-2010-017.

P. E. Shrout and J. L. Fleiss. Intraclass correlations: Uses in assessing rater reliability. *Psychological Bulletin*, 86(2):420–428, 1979. doi: 10.1037/0033-2909.86.2.420.

D. J. Stekhoven and P. Bühlmann. MissForest—non-parametric missing value imputation for mixed-type data. *Bioinformatics*, 28(1):112–118, 2012. doi: 10.1093/bioinformatics/btr597.

S. van Buuren. *Flexible Imputation of Missing Data*. CRC Press, Boca Raton, FL, 2nd edition, 2018. URL `https://stefvanbuuren.name/fimd/`.

M. P. J. van der Loo. The **stringdist** package for approximate string matching. *The R Journal*, 6(1): 111–122, 2014. doi: 10.32614/RJ-2014-011.

W. E. Winkler. String comparator metrics and enhanced decision rules in the Fellegi-Sunter model of record linkage. In *Proceedings of the Section on Survey Research Methods, American Statistical Association*, pages 354–359, 1990.

M. N. Wright and A. Ziegler. **ranger**: A fast implementation of random forests for high dimensional data in C++ and R. *Journal of Statistical Software*, 77(1):1–17, 2017. doi: 10.18637/jss.v077.i01.